МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
«Южно-Уральский государственный университет
(национальный исследовательский университет)»
Институт естественных и точных наук
Кафедра системного программирования

ОТЧЕТ

о выполнение практической работы №1

по дисциплине

«Основы интеллектуального анализа данных»

Проверил:

доцент кафедры СП, к. ф.-м. наук

_____Б.А. Марков

Автор работы:
студент группы ЕТ-401

_____А.О. Жалий

Челябинск, 2026 г.

ОГЛАВЛЕНИЕ

1. ФОРМУЛИРОВКА ЗАДАНИЯ 3
1.1. Поиск частных наборов 3
1.2. Поиск ассоциативных правил 3
2. ИСХОДНЫЙ КОД 5
3. ТЕОРИЯ 6
3.1. Частые наборы и поддержка 6
3.2. Ассоциативные правила 6
4. РЕАЛИЗАЦИЯ 8
4.1. Общая структура программы 8
4.2. Алгоритм Apriori 8
4.3. Генерация ассоциативных правил 9
4.4. Реализация FP-Growth 9
5. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ 12
5.1. Задание 1. Варьирование порога поддержки 12
5.2. Задание 2. Варьирование порога достоверности 14
5.3. Задание 3. Результаты FP-Growth 15
6. ВЫВОДЫ 18

1. ФОРМУЛИРОВКА ЗАДАНИЯ

1.1. Поиск частных наборов

Разработать программу, которая выполняет поиск частых наборов объектов в заданном наборе данных с помощью алгоритма Apriori (или одной из его модификаций). Список результирующих наборов должен содержать как наборы, так и значение поддержки для каждого набора. Параметрами программы являются:

- набор данных;
- порог поддержки;
- способ упорядочивания результирующего списка наборов (по убыванию значения поддержки или лексикографическое).

Провести эксперименты на наборе данных baskets.csv (сведения о покупках в супермаркете), варьируя пороговое значение поддержки (например: 1%, 3%, 5%, 10%, 15%).

Выполнить визуализацию результатов экспериментов в виде диаграмм:

- сравнение быстродействия на фиксированном наборе данных при изменяемом пороге поддержки;
- количество частых наборов объектов различной длины на фиксированном наборе данных при изменяемом пороге поддержки.

1.2. Поиск ассоциативных правил

Доработать программу из задания 1 так, чтобы она также выполняла поиск ассоциативных правил. Список результирующих правил должен выдаваться в удобочитаемом виде (антецедент → консеквент) с указанием поддержки и достоверности каждого правила. Дополнительные параметры программы: порог достоверности, способ упорядочивания результирующего списка наборов (по убыванию значения поддержки или лексикографическое).

Провести эксперименты на наборе данных из задания 1. Зафиксировать порог поддержки (например, 10%), варьировать порог достоверности (например, от 70% до 95% с шагом 5%).

Выполнить визуализацию:

- сравнение быстродействия поиска правил на фиксированном наборе данных при изменяемом пороге достоверности;
- общее количество найденных правил на фиксированном наборе данных при изменяемом пороге достоверности.

Подготовить список правил, в которых антецедент и консеквент суммарно включают не более семи объектов. Проанализировать и изложить содержательный смысл полученного результата.

2. ИСХОДНЫЙ КОД

Исходный код (Python), набор данных baskets.csv (7501 покупателская корзина, 115 наименований товаров), результаты экспериментов и диаграммы доступны по ссылке: https://github.com/TWorkerFrog/FindTemplate

3. ТЕОРИЯ

3.1. Частые наборы и поддержка

Пусть есть база транзакций, где каждая транзакция – это множество товаров, купленных вместе. Набор объектов – это некоторое подмножество множества всех товаров. Поддержка набора показывает, какая доля транзакций содержит этот набор. Если поддержка не меньше заданного порога, набор называется частым. Задача поиска частых наборов найти все такие наборы в базе.

Apriori – это классический итеративный алгоритм поиска частых наборов, предложенный Агравалом и Шрикантом в 1994 году. Он основан на свойстве антимонотонности: если набор является частым, то все его непустые подмножества тоже частые; и наоборот, если набор не является частым, то все его надмножества тоже не могут быть частыми. Это свойство позволяет существенно сократить пространство поиска и не рассматривать заведомо бесперспективные комбинации.

Алгоритм работает по уровням. Сначала подсчитывается поддержка каждого отдельного товара и формируется уровень частых 1-элементных наборов L1. Затем из частых наборов предыдущего уровня $L(k-1)$ генерируются кандидаты $C(k)$ с помощью двух операций: join – соединение пар наборов, имеющих общий префикс длины $k-2$, и prune – отсечение тех кандидатов, у которых хотя бы один $(k-1)$-поднабор не является частым. После этого выполняется сканирование базы данных и подсчёт поддержки оставшихся кандидатов; те, что прошли порог, образуют уровень $L(k)$. Процесс повторяется, пока очередной уровень не окажется пустым.

3.2. Ассоциативные правила

Из найденных частых наборов строятся ассоциативные правила вида $X \rightarrow Y$, где X – антецедент (условие), а Y – консеквент (следствие). Каждое правило характеризуется тремя метриками. Поддержка (support) – доля корзин, содержащих одновременно и X, и Y. Достоверность (confidence) – доля

корзин с X, в которых дополнительно встречается Y; это условная вероятность увидеть Y при условии, что в корзине есть X. Лифт (lift) показывает, во сколько раз вероятность покупки Y возрастает при наличии X в корзине по сравнению с безусловной вероятностью Y. Если лифт больше единицы, товары положительно связаны, если равен единице, независимы, если меньше единицы, связаны отрицательно. Правило считается интересным, если его поддержка не меньше заданного порога поддержки, а достоверность не меньше заданного порога достоверности.

4. РЕАЛИЗАЦИЯ

4.1. Общая структура программы

Программа реализована на языке Python с использованием только стандартной библиотеки (collections, itertools) и matplotlib для визуализации. Она состоит из двух логических частей, соответствующих двум заданиям.

Набор данных загружается из файла baskets.csv: одна транзакция в строке, товары перечислены через запятую. Порог поддержки, порог достоверности и способ упорядочивания результатов задаются пользователем при запуске программы.

4.2. Алгоритм Apriori

Реализован классический Apriori, работающий по уровням.

Шаг 1. Поиск L1. Подсчитывается поддержка каждого отдельного товара – для этого выполняется одно сканирование базы. Товары, чья поддержка не меньше заданного порога, образуют уровень частых 1-элементных наборов L1.

Шаг 2. Генерация кандидатов C(k). Из частых наборов предыдущего уровня L(k−1) формируются кандидаты размера k. Для этого используется двойной цикл по парам наборов: если объединение двух наборов даёт ровно k элементов, кандидат рассматривается дальше. Затем выполняется операция prune: проверяется, что все (k−1)-поднаборы кандидата присутствуют в L(k−1). Если хотя бы один поднабор не является частым – кандидат отбрасывается. Это прямое применение свойства антимонотонности Apriori.

Шаг 3. Подсчёт поддержки. Для каждого кандидата подсчитывается, сколько транзакций его содержат. Это второе (и последнее на данном уровне) сканирование базы.

Шаг 4. Формирование L(k). Кандидаты, чья поддержка прошла порог, образуют уровень L(k). Процесс повторяется с k = k + 1, пока очередной уровень не окажется пустым.

Результирующий список содержит все частые наборы с указанием поддержки и упорядочивается по убыванию поддержки либо лексикографически – в зависимости от выбора пользователя.

4.3. Генерация ассоциативных правил

После того как все частые наборы найдены, из них строятся ассоциативные правила. Для каждого частого набора размера ≥ 2 перебираются все возможные разбиения на антецедент X и консеквент Y. Для каждого разбиения вычисляются три метрики:

support – доля транзакций, содержащих и X, и Y одновременно;

confidence – доля транзакций с X, в которых дополнительно встречается Y;

lift – во сколько раз вероятность покупки Y возрастает при наличии X в корзине.

Правило попадает в итоговый список, если его достоверность не меньше заданного порога. Результаты выводятся в удобочитаемом виде: {антецедент} → {консеквент} с указанием поддержки, достоверности и лифта.

4.4. Реализация FP-Growth

Дополнительно к классическому Apriori была реализована его модификация: алгоритм FP-Growth (Frequent Pattern Growth). В отличие от Apriori, который на каждом уровне генерирует кандидатов и многократно сканирует базу данных, FP-Growth сжимает исходные транзакции в специальную структуру – FP-дерево – и извлекает частые наборы непосредственно из него.

Алгоритм состоит из двух этапов. На первом этапе выполняется сканирование базы и подсчёт частоты каждого товара; товары, не прошедшие порог поддержки, отбрасываются. На втором этапе строится FP-дерево: каждая транзакция вставляется в дерево, при этом товары внутри

транзакции сортируются по убыванию частоты – это обеспечивает компактность дерева и объединение общих префиксов. Узлы дерева хранят название товара, счётчик вхождений и ссылки на родителя и детей; одноимённые узлы связаны в цепочку через header table.

Реализация включает:

класс FPNode – узел дерева с полями name, count, parent, children, link;

функцию build_fp_tree – построение дерева и header table;

функцию plot_fp_tree – визуализацию дерева через matplotlib с ограничением глубины и числа детей для читаемости.

4.5. ECLAT

Дополнительно был реализован алгоритм ECLAT (Equivalence Class Clustering and bottom-up Lattice Traversal) ещё одна модификация Apriori для поиска частых наборов. В отличие от Apriori, который хранит данные в горизонтальном формате (транзакция → список товаров) и сканирует базу на каждом уровне, ECLAT использует вертикальный формат: каждому товару сопоставляется множество идентификаторов транзакций (TID-list), в которых он встречается. Поддержка набора в ECLAT вычисляется не сканированием базы, а пересечением TID-списков входящих в него товаров.

Алгоритм работает в глубину (DFS), а не по уровням (BFS), как Apriori. На первом шаге выполняется одно сканирование базы и строится вертикальный формат данных; товары, не прошедшие порог поддержки, отбрасываются. На втором шаге запускается рекурсивная процедура: для каждого товара формируется текущий префикс, а для всех последующих товаров вычисляется пересечение TID-списков. Если длина пересечения не меньше порога, товар добавляется в следующий уровень, и рекурсия продолжается. Кандидаты явно не генерируются отсечение происходит автоматически: если пересечение меньше порога, ветка поиска не развивается.

Реализация включает:

- функцию build_tid_lists построение вертикального формата данных (TID-list для каждого товара);
- функцию eclat рекурсивный поиск частых наборов через пересечение TID-списков;
- группировку найденных наборов по размеру и вывод с поддержкой.

Параметры программы: путь к набору данных, порог поддержки. Результат все частые наборы с указанием поддержки, сгруппированные по длине.

5. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

5.1. Задание 1. Варьирование порога поддержки

Эксперименты проводились на наборе baskets.csv. Порог поддержки варьировался от 1% до 15%. Результаты приведены в таблице 1.

Таблица 1 – Результаты эксперимента (задание 1)

Порог под-держки	Длина 1	Длина 2	Длина 3	Всего	Время
1%	76	74	170	17	≈ 0,9–1,8 с
3%	226	36	19	–	≈ 170 мс
5%	376	25	3	–	≈ 90 мс
10%	751	7	–	–	≈ 16 мс
15%	1126	5	–	–	≈ 15 мс

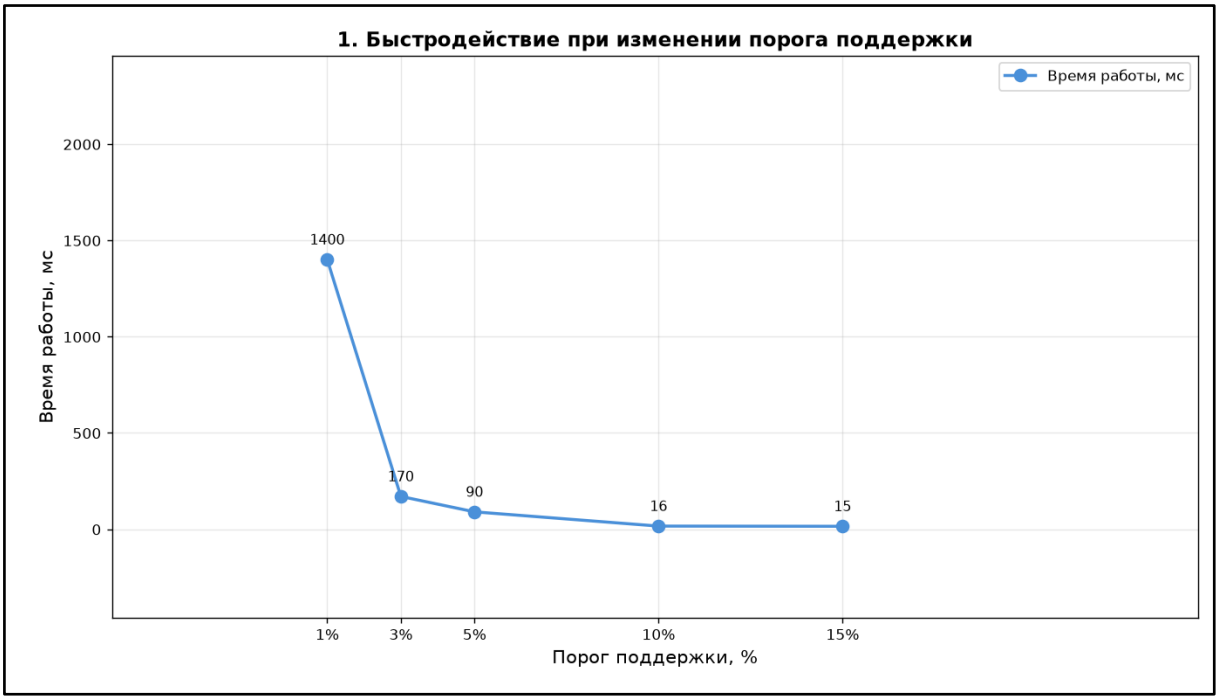


Рисунок 1 – Быстродействие при изменении порога поддержки

=

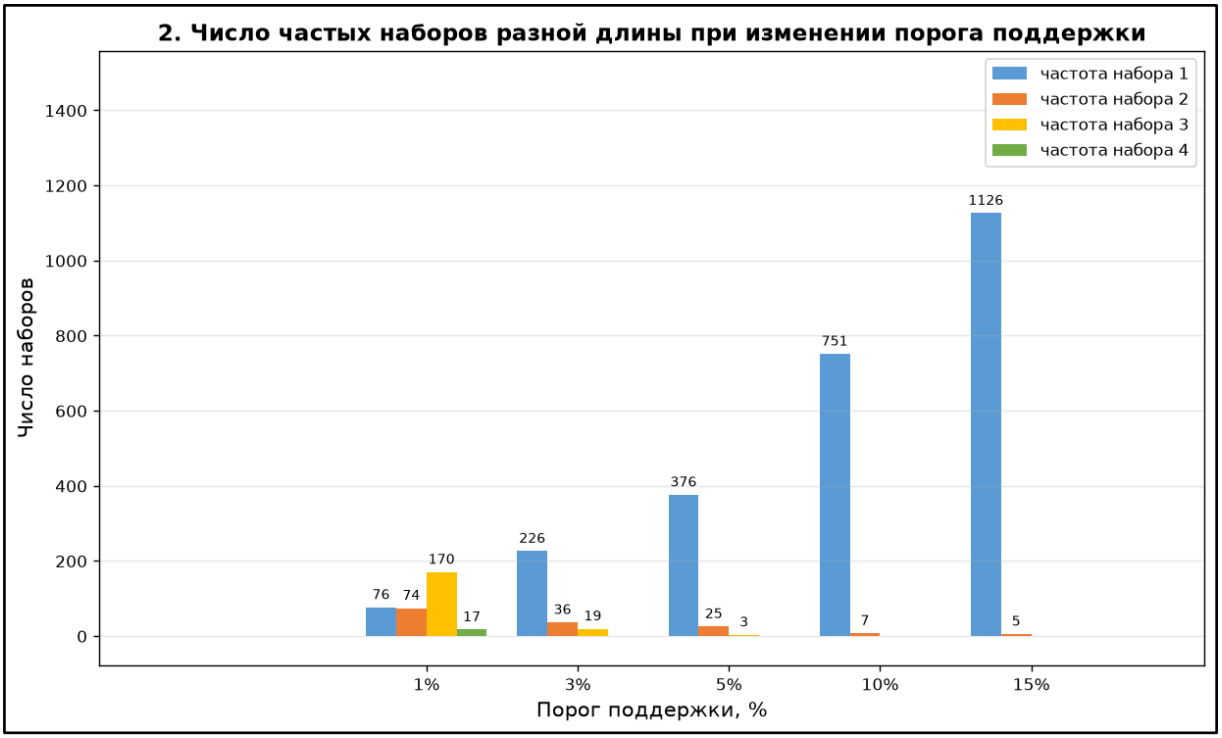


Рисунок 2 – Число частых наборов разной длины при изменении порога поддержки

С ростом порога количество частых наборов резко сокращается: с 261 при пороге 1% до 5 при пороге 15%. Наборы длины 3 появляются только при пороге 1%: чтобы три товара встретились вместе минимум в 76 корзинах, каждый из них должен приобретаться очень часто. При порогах 10% и 15% выживают только одиночные товары с максимальной встречаемостью (минеральная вода – 1788 корзин, макароны – 1410, яйца – 1348, картофель-фри – 1282, шоколад – 1229); ни одна пара не достигает требуемой совместной встречаемости.

Время работы растёт нелинейно при снижении порога: с ~15 мс при 15% до ~1 с при 1%, то есть более чем в 60 раз. Причина – каскадный эффект: чем ниже порог, тем больше частых одиночных товаров, из них собирается больше пар-кандидатов, выжившие пары порождают тройки, и объём работы на каждом уровне умножается. Именно поэтому порог поддержки является главным средством управления трудоёмкостью алгоритма.

Существенная часть кандидатов отсекается на этапе генерации без обращения к базе: если хотя бы один (k−1)-поднабор кандидата не является частым, кандидат не может быть частым и не проверяется. Например, кандидат {масло, молоко, хлеб} не генерируется, поскольку пара {масло, молоко} не прошла порог. Это сокращает пространство поиска по сравнению с полным перебором всех возможных комбинаций товаров.

5.2. Задание 2. Варьирование порога достоверности

Эксперименты проводились при фиксированной поддержке 10%. Порог достоверности варьировался от 70% до 95% с шагом 5%.

При повышении порога достоверности количество правил монотонно уменьшается: при 70% правил много, при 95% остаются только самые сильные. Время работы меняется слабо, так как основной вклад вносит поиск частых наборов, который не зависит от порога достоверности; генерация правил – быстрая операция. Существенное сокращение количества правил начинается уже на пороге 80–85%, а при 95% остаются единичные правила с очень высокой условной вероятностью.

Таблица 2 – Примеры правил с наибольшим lift (порог поддержки 1%, confidence ≥ 0,3):

Правило	Support	Confidence	Lift
{травы и перец} → {говяжий фарш}	1,6%	32,3%	3,29
{замороженные овощи, минеральная вода} → {молоко}	1,1%	31,0%	2,39
{суп} → {молоко}	1,5%	30,1%	2,32

{говяжий фарш} → {макароны}	4,0%	41,0%	2,18

Метрика lift показывает силу связи: значение 3,29 для правила {травы и перец} → {говяжий фарш} означает, что при наличии в корзине специй вероятность покупки фарша возрастает в 3,3 раза относительно средней. Lift > 1 указывает на положительную связь товаров (совместная покупка), lift ≈ 1 – на независимость, lift < 1 – на взаимозаменяемость. Отметим, что на порогах 3% и выше правила не строятся, так как там не выживает ни одной частой пары, а правило можно построить только из набора длины не менее 2.

5.3. Задание 3. Результаты FP-Growth

Эксперименты проводились на том же наборе baskets.csv, что и для Apriori. FP-дерево строилось при различных порогах поддержки; для визуализации выбиралась ограниченная глубина, чтобы дерево оставалось читаемым.

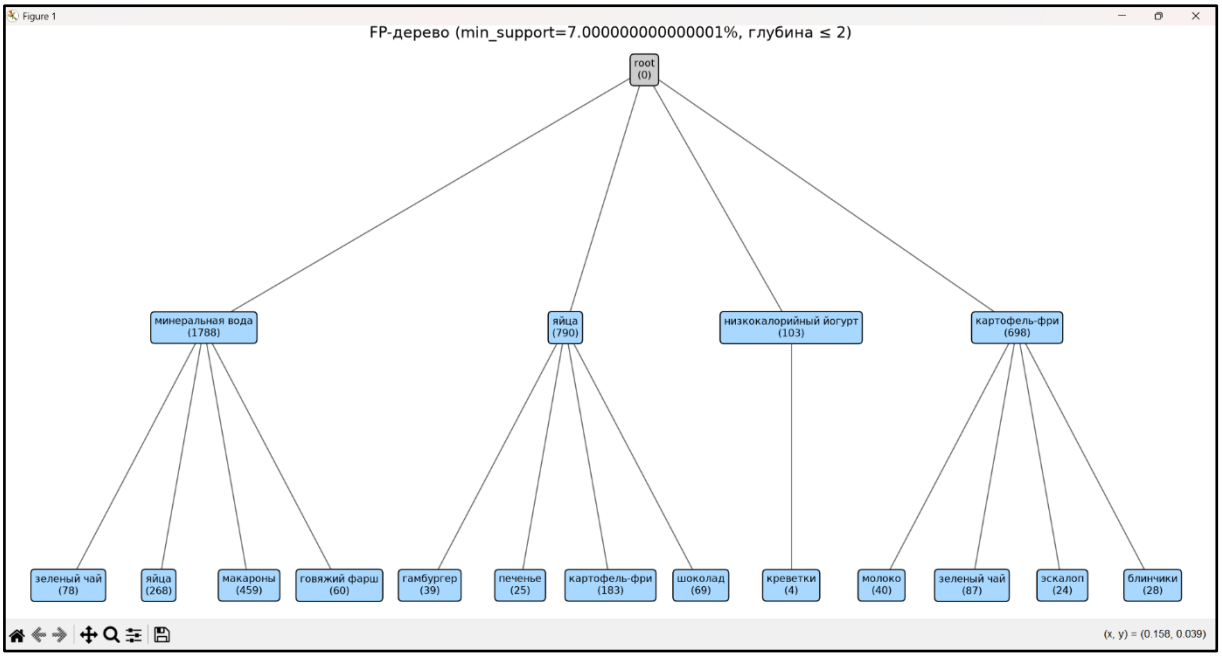


Рисунок 3 – FP-дерево, построенное на наборе baskets.csv

FP-дерево компактно представляет частые наборы: общие префиксы

транзакций разделяются между ветвями, что позволяет избежать повторного хранения одних и тех же товаров. Чем выше порог поддержки, тем меньше узлов остаётся в дереве; при порогах 10–15% дерево содержит только самые частые товары (минеральная вода, макароны, яйца, картофель-фри, шоколад). Счётчики в узлах показывают, сколько транзакций проходят через данный узел.

5.4 Задание 4. Результаты ECLAT

Эксперименты проводились на том же наборе baskets.csv, что и для Apriori и FP-Growth. Порог поддержки варьировался от 1% до 15%

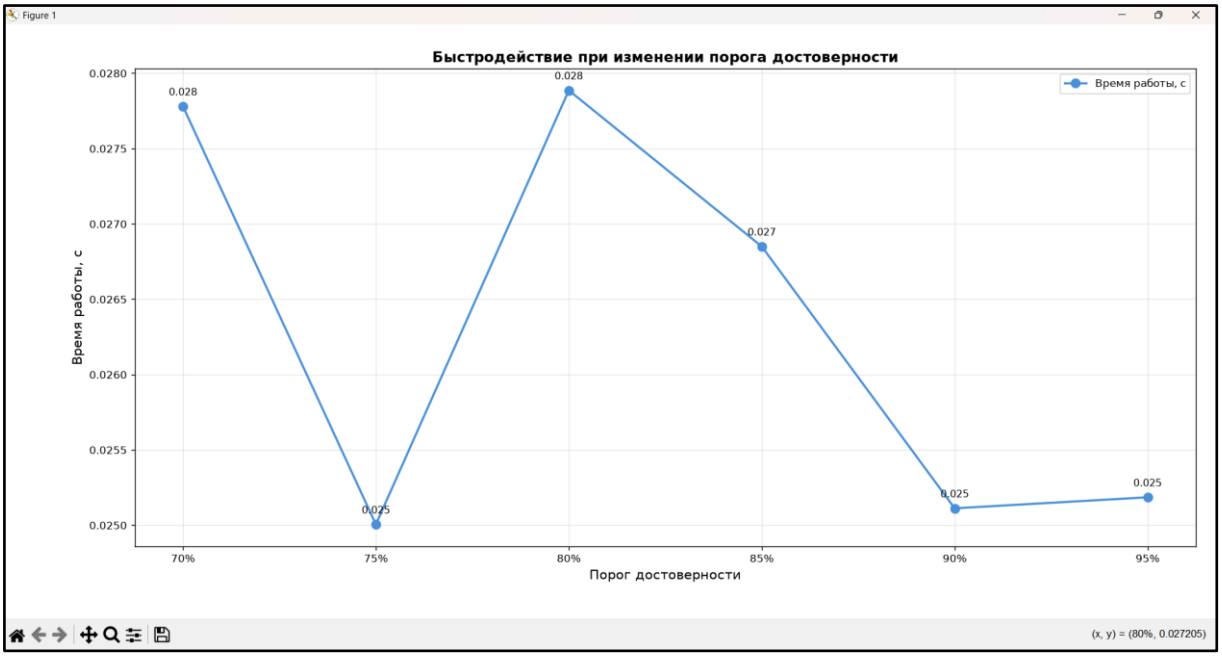


Рисунок 4 – Быстродействие при изменении порога достоверности

На графике видно, что время работы слабо меняется при изменении порога достоверности. Это объясняется тем, что основной вклад вносит поиск частых наборов (Apriori), который не зависит от порога достоверности, а генерация правил быстрая операция.

Результаты ECLAT совпадают с результатами Apriori: оба алгоритма находят одни и те же частые наборы, поскольку решают одну задачу. Разница в скорости работы. ECLAT выполняет только одно сканирование базы

при построении TID-списков, а дальнейший поиск идёт через пересечение множеств, что делает его быстрее Apriori на плотных данных. Однако TID-списки занимают больше памяти, и при большом числе частых товаров пересечения становятся дорогими.

6. ВЫВОДЫ

В ходе работы были реализованы алгоритм Apriori для поиска частых наборов объектов и генератор ассоциативных правил с расчётом поддержки, достоверности и лифта. Программа позволяет задавать порог поддержки, порог достоверности и способ упорядочивания результатов.

Эксперименты на наборе baskets.csv показали, что при повышении порога поддержки количество частых наборов резко сокращается, а время работы уменьшается. При пороге 1% алгоритм находит сотни наборов и работает заметно дольше из-за комбинаторного взрыва кандидатов; при пороге 15% остаются только самые частые одиночные товары. Это согласуется со свойством антимонотонности: чем выше порог, тем меньше наборов проходят фильтр, и тем меньше кандидатов генерируется на следующих уровнях.

При фиксированной поддержке 10% повышение порога достоверности от 70% до 95% приводит к монотонному уменьшению количества правил, при этом время работы меняется слабо – основной вклад вносит поиск частых наборов, а не генерация правил.

Наиболее сильные правила связаны с комбинациями фастфуда (картофель-фри → гамбургер), базовых продуктов (макароны, яйца → молоко), сладостей (шоколад, молоко → печенье). Лифт больше единицы для большинства правил говорит о положительной связи между товарами: их совместная покупка не случайна. Такие правила могут использоваться для мерчендайзинга, рекомендательных систем, формирования акционных наборов и анализа ассортимента.

Дополнительно был реализован алгоритм FP-Growth модификация Apriori, использующая FP-дерево вместо генерации кандидатов. Сравнение двух подходов показывает:

Apriori проще в реализации, но требует многократного сканирования базы данных и явной генерации кандидатов, что замедляет работу при низких порогах поддержки.

FP-Growth выполняет только два сканирования базы (подсчёт частот и построение дерева) и не генерирует кандидатов, что делает его быстрее на больших наборах данных. Однако FP-дерево требует дополнительной памяти и сложнее в реализации.

ECLAT использует вертикальный формат и пересечение TID-списков; быстрее Apriori на плотных данных, но требует памяти под TID-списки и становится медленным на разреженных данных.

Оба алгоритма дают одинаковый результат множество частых наборов, что подтверждает корректность реализаций. Выбор алгоритма зависит от задачи: для небольших наборов удобнее Apriori, для больших FP-Growth